UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto “CodeBreaker - Aprende Lenguajes de Programación “

Curso: PATRONES DE SOFTWARE

Docente: ING. PATRICK JOSE CUADROS QUIROGA

Integrantes:

Rosas Chambilla, Jefferson	***2021072618***
Challo Coaquera Alexsander Wilson	***2017057578***
Huaman Rivera, Roberto Carlos	**2021071077**

Tacna – Perú

2026

CodeBreaker - Aprende Lenguajes de Programación
Documento Informe de Factibilidad

Versión *{1.00}*

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	MPV	ELV	ARV	10/4/2026	Versión Original

ÍNDICE GENERAL

1. Descripción del Proyecto.....6
1.1 Nombre del proyecto.....6
1.2 Duración del proyecto.....6
1.3 Descripción.....7
1.4 Objetivos.....8
1.4.1 Objetivo General.....8
1.4.2 Objetivos Específicos.....8
2. Riesgos.....9
2.1 Riesgos Técnicos.....9
2.2 Riesgos de Gestión y Cronograma (Hitos).....10
2.3 Riesgos de Usuario y Contenido.....10
2.4 Riesgos Legales y de Seguridad.....10
3. Análisis de la Situación actual.....11
3.1 Planteamiento del problema.....11
3.1.1 Problema General.....11
3.1.2 Problemas Específicos.....11
3.2 Consideraciones de hardware y software.....12
3.2.1 Inventario disponible.....12
3.2.2 Tecnología propuesta.....12
4. Estudio de Factibilidad.....13
4.1 Factibilidad Técnica.....13
4.1.1 Configuración de Hardware y Recursos Físicos.....13
4.1.2 Stack de Software y Aplicabilidad.....13
4.1.3 Infraestructura de Red y Disponibilidad.....14
4.1.4 Metodología de Implementación Técnica.....15
4.2 Factibilidad Económica.....16
4.2.1 Costos Generales.....17
4.2.2 Costos operativos durante el desarrollo.....17
4.2.3 Costos del ambiente.....18
4.2.4 Costos de personal.....18
4.2.5 Costos totales del desarrollo del sistema.....18
4.2.6 Evaluación Financiera.....19
4.2.7 Análisis de Sensibilidad.....19
4.3 Factibilidad Operativa.....19
4.3.1 Estructura Organizacional y Roles del Equipo.....19
4.3.2 Análisis de Aceptación del Usuario.....20

4.3.3 Procesos de Soporte y Mantenimiento..... 21
4.3.4 Matriz de Impacto Operativo.....21
4.4 Factibilidad Legal.....22
4.4.1 Protección de Datos Personales..... 22
4.4.2 Propiedad Intelectual y Derechos de Autor.....22
4.4.3 Responsabilidad Civil y Ejecución de Código (Sandbox)..... 22
4.4.4 Cumplimiento Tributario y Formalización.....23
4.5 Factibilidad Social.....23
4.5.1 Democratización del Aprendizaje Técnico..... 23
4.5.2 Reducción de la Curva de Frustración..... 24
4.5.3 Alineación con los Objetivos de Desarrollo Sostenible (ODS).....24
4.6 Factibilidad Ambiental..... 24
4.6.1 Reducción del Consumo de Recursos Físicos.....25
4.6.2 Eficiencia Energética y Cloud Computing.....25
4.6.3 Gestión de Residuos Digitales.....25
5. Análisis Financiero..... 26
5.1 Justificación de la Inversión..... 27
5.1.2 Criterios de Inversión.....28
5.1.2.1 Relación Beneficio/Costo (B/C)..... 28
5.1.2.2 Valor Actual Neto (VAN).....28
5.1.2.3 Tasa Interna de Retorno (TIR).....29
6. Conclusiones..... 29

Informe de Factibilidad

1. Descripción del Proyecto

CodeBreaker es una plataforma web educativa diseñada para democratizar y gamificar el aprendizaje de la programación mediante el enfoque “Learning by doing”. Su proposito central es eliminar la barrera de entrada que supone la configuración de entornos locales complejos, ofreciendo un editor de codigo en tiempo real accesible desde cualquier navegador.

1.1 Nombre del proyecto

CodeBreaker - Aprende Lenguajes de Programación

1.2 Duración del proyecto

Fase	Actividad Principal	Duración Estimada	Fechas Tentativas
Carga de Contenido	Estructuración y carga de los primeros 50 desafíos de programación (Python/JS).	7 días	19 may – 25 may
Desarrollo Frontend	Integración del editor de código dinámico (resaltado de sintaxis y autocompletado).	10 días	09 may – 18 may
Lógica de Evaluación	Creación del algoritmo que valida si el código del alumno cumple con el reto planteado.	8 días	01 may – 08 may
Hito 1: MVP Técnico	Primera Presentación: Demostración de ejecución de código segura y prototipo funcional.	Hito	30 de abril
Motor de Ejecución	Desarrollo del Sandbox (entorno aislado) para procesar código de forma segura.	18 días	08 abr – 25 abr
Infraestructura	Configuración de servidores, entornos de desarrollo y base de datos inicial.	7 días	01 abr – 07 abr
Diseño UI/UX	Creación de wireframes y diseño de interfaz del editor de código y perfil del usuario.	5 días	27 mar – 31 mar
Requerimientos	Documentación de Requisitos Funcionales, No Funcionales y casos de uso.	4 días	23 mar – 26 mar

Hito Final	Tercera Presentación: Sustentación final de CodeBreaker 100% operativo.	Hito	30 de junio
Despliegue y Cierre	Configuración de dominio final y sustentación del proyecto terminado.	2 días	29 jun – 30 jun
Documentación Final	Elaboración de manuales técnicos, de usuario y reporte de factibilidad final.	5 días	24 jun – 28 jun
Pruebas de Usuario	Pruebas Beta con estudiantes reales para ajustar la dificultad de los ejercicios.	5 días	19 jun – 23 jun
Seguridad y QA	Pruebas de penetración (anti-hackeo) y optimización de carga de usuarios concurrentes.	8 días	11 jun – 18 jun
Gamificación	Implementación de sistema de niveles, medallas, puntos y tabla de posiciones (Ranking).	10 días	01 jun – 10 jun
Hito 2: Beta Educativa	Segunda Presentación: Plataforma con retos, corrección automática y dashboard de progreso.	Hito	30 de mayo
Iniciación	Definición de objetivos, benchmarking de plataformas educativas y selección de stack tecnológico.	8 días	15 mar – 22 mar

1.3 Descripción

Nombre del proyecto: CodeBreaker – Plataforma Interactiva para el Aprendizaje de Lenguajes de Programación.

Duración del proyecto: Mediados de Marzo 2026 – Junio 2026 (Desarrollo integral y sustentación por hitos mensuales).

Resumen Ejecutivo: CodeBreaker es una aplicación web educativa diseñada para democratizar y gamificar el aprendizaje de la programación. A diferencia de los métodos de lectura pasiva, CodeBreaker se centra en el "Aprender Haciendo" (Learning by Doing), integrando un editor de código en tiempo real que permite a los estudiantes escribir, compilar y validar sus ejercicios de forma inmediata dentro del navegador.

El Problema que resuelve: Muchos estudiantes principiantes abandonan la programación debido a la complejidad de configurar entornos de desarrollo locales (instalación de compiladores, variables de entorno, IDEs pesados). CodeBreaker elimina esta barrera de entrada proporcionando un entorno pre-configurado, seguro y accesible desde cualquier dispositivo con internet.

Componentes Clave:

- Sandbox Seguro: Un motor de ejecución aislado (basado en contenedores) que procesa el código del usuario sin comprometer la seguridad del servidor.
- Evaluación Automatizada: Un sistema de "Juez en Línea" que analiza la lógica del estudiante y ofrece retroalimentación instantánea.
- Rutas de Aprendizaje: Módulos estructurados que van desde conceptos básicos (variables, bucles) hasta estructuras de datos avanzadas.
- Capa de Gamificación: Implementación de un sistema de recompensas (puntos, niveles y medallas) para mantener la motivación y reducir la tasa de deserción.

Contexto y Alcance: El proyecto se desarrolla bajo un modelo de entrega por hitos mensuales (Abril, Mayo y Junio), asegurando que cada fase técnica sea validada antes de pasar a la siguiente. Inicialmente, la plataforma soportará lenguajes de alta demanda como Python y JavaScript, con una arquitectura escalable para añadir más lenguajes en el futuro.

1.4 Objetivos

1.4.1 Objetivo General

Desarrollar una plataforma web educativa e interactiva que facilite el aprendizaje de lenguajes de programación mediante un sistema de ejecución de código en tiempo real y retroalimentación automatizada, mejorando la retención de conocimientos y la motivación del estudiante entre marzo y junio de 2026.

1.4.2 Objetivos Específicos

Para alcanzar el objetivo general, se plantean las siguientes metas técnicas y operativas:

I. Implementar un entorno de ejecución seguro (Sandbox): Diseñar una arquitectura basada en contenedores o aislamiento de procesos que permita ejecutar el código escrito por el usuario de forma segura, evitando vulnerabilidades en el servidor.

II. Desarrollar un editor de código interactivo: Integrar una interfaz de edición con resaltado de sintaxis y autocompletado que emule la experiencia de un entorno de desarrollo profesional dentro del navegador.

III. Diseñar un motor de evaluación automática: Crear algoritmos de validación que comparen el "output" del código del estudiante con los resultados esperados, proporcionando correcciones instantáneas.

IV. Establecer rutas de aprendizaje estructuradas: Organizar el contenido educativo en módulos progresivos (básico, intermedio y avanzado) para los lenguajes iniciales (Python y JavaScript).

V. Integrar una capa de gamificación: Desarrollar un sistema de recompensas que incluya puntos, medallas y niveles de progreso para aumentar el compromiso (engagement) de los usuarios.

VI. Garantizar la escalabilidad y disponibilidad: Configurar el sistema para soportar una carga creciente de usuarios concurrentes mediante el despliegue en la nube y optimización de recursos.

VII. Validar la usabilidad mediante hitos: Cumplir con las tres presentaciones mensuales programadas (abril, mayo y junio) para ajustar el sistema según el feedback técnico y pedagógico recibido.

2. Riesgos

El proyecto identifica cuatro categorias criticas de riesgos:

- Técnicos: Ejecución de código malicioso y consumo excesivo de recursos (bucles infinitos). Se mitigarán mediante Sandboxing con Docker y configuración de Timeouts.
- Gestión: Retrasos en hitos mensuales y subestimación de la curva de aprendizaje. Se aplicará metodología Scrum y priorización de lenguajes sencillos.
- Usuario: Frustración por mala retroalimentación o deserción. Se implementarán mensajes de error personalizados y gamificación.
- Legales: Incumplimiento de la Ley de Protección de Datos (Perú). Se utilizará encriptación bcrypt y políticas de privacidad claras.

2.1 Riesgos Técnicos

Ejecución de Código Malicioso: Que un usuario intente enviar comandos para borrar archivos del servidor o acceder a la base de datos a través del editor.

- Mitigación: Implementar Sandboxing estricto mediante el uso de contenedores aislados gestionados a través de la API de Piston. El backend desarrollado en Node.js actuará como intermediario, validando las peticiones mediante una arquitectura asíncrona y orquestando la ejecución en entornos sin privilegios de red ni acceso al sistema de archivos del servidor principal.

Consumo Excesivo de Recursos (Infinite Loops): Que un código con un bucle infinito sature el CPU del servidor.

- Mitigación: Configurar un Timeout de ejecución gestionado desde el servicio de Node.js mediante promesas con tiempo de expiración y parámetros de la API de Piston. Asimismo, establecer restricciones

estrictas de memoria RAM y CPU por cada proceso de ejecución de código para evitar el agotamiento de recursos del servidor.

Incompatibilidad del Entorno de Desarrollo: Discrepancias en el funcionamiento del sistema debido a diferencias en las versiones de Node.js o módulos de npm entre los desarrolladores.

- Mitigación: Uso de un entorno de ejecución estandarizado con Node.js (LTS) y gestión de dependencias mediante npm o yarn. Se empleará un archivo package-lock.json y archivos de configuración de entorno (.env) para asegurar que todas las librerías y versiones de los módulos sean idénticas en todos los entornos de desarrollo, evitando el error de "funciona en mi máquina".

2.2 Riesgos de Gestión y Cronograma (Hitos)

- Incumplimiento de Entregas Mensuales: Retrasos en el desarrollo del motor de ejecución que afecten la presentación de finales de abril.
  - Mitigación: Aplicar metodología Scrum con entregables mínimos viables (MVP) para asegurar que siempre haya algo que mostrar.
- Subestimación de la Curva de Aprendizaje: Dificultad técnica imprevista al integrar la ejecución de múltiples lenguajes en el Sandbox (ej. gestionar dependencias de JavaScript frente a la simplicidad de Python).
  - Mitigación: Priorizar el motor de ejecución de Python para el primer hito de abril/mayo, dejando la integración completa de JavaScript para la segunda fase, asegurando así una base estable antes de añadir complejidad.

2.3 Riesgos de Usuario y Contenido

- Baja Calidad en la Retroalimentación: Que el sistema diga que el código está "Mal" pero no explique por qué, frustrando al alumno.
  - Mitigación: Desarrollar un sistema de mensajes de error personalizados que traduzcan errores técnicos a lenguaje sencillo.
- Deserción por Falta de Motivación: Que los retos sean demasiado fáciles o muy difíciles.
  - Mitigación: Implementar la fase de Gamificación (Hito de Junio) para mantener el interés mediante niveles y medallas.

2.4 Riesgos Legales y de Seguridad

- Protección de Datos Personales: Incumplimiento de la Ley N° 29733 (Ley de Protección de Datos Personales en Perú) al almacenar correos y progresos de estudiantes.
  - Mitigación: Encriptación de contraseñas con bcrypt y creación de políticas de privacidad claras.

- Plagio de Contenido Educativo: Riesgo de usar ejercicios protegidos por derechos de autor de otras plataformas.
    - Mitigación: Crear contenido propio y ejercicios originales basados en la currícula diseñada en marzo.

3. Análisis de la Situación actual

En la actualidad, el aprendizaje de la programación se ha vuelto una competencia esencial en el mercado laboral. Sin embargo, en el entorno educativo (específicamente en instituciones de formación inicial o para autodidactas), el proceso de enseñanza presenta las siguientes características:

- Entornos de configuración complejos: El aprendizaje se detiene antes de empezar debido a la necesidad de instalar compiladores y lenguajes (como Python o JavaScript), configurar variables de entorno y elegir un IDE. Esto genera una "barrera de entrada" técnica que provoca frustración y abandono temprano. CodeBreaker elimina este obstáculo al ofrecer un entorno de ejecución basado en la nube, permitiendo al estudiante programar instantáneamente desde el navegador.
- Teoría vs. Práctica: Muchos cursos se basan en videos o libros donde el estudiante es un espectador pasivo. Existe una falta de herramientas que permitan la práctica inmediata sin salir del navegador.
- Retroalimentación retardada: En un entorno tradicional, un estudiante escribe código y debe esperar a que un tutor lo revise para saber si cometió un error lógico, lo que frena el ritmo de aprendizaje.
- Baja motivación: La falta de incentivos visuales o sistemas de recompensa (gamificación) hace que el estudio de la sintaxis pura sea percibido como tedioso y difícil.

3.1 Planteamiento del problema

3.1.1 Problema General

¿De qué manera el desarrollo de una plataforma interactiva basada en la ejecución de código en tiempo real y gamificación puede reducir las barreras de entrada y mejorar el nivel de aprendizaje en los estudiantes principiantes de programación?

3.1.2 Problemas Específicos

- Barrera Técnica: ¿Cómo eliminar la necesidad de configuraciones locales complejas para que el estudiante pueda programar desde cualquier dispositivo de forma inmediata?
- Validación de Conocimientos: ¿De qué forma se puede proporcionar una calificación automática y precisa que permita al alumno corregir sus errores de lógica en tiempo real?

- Retención y Motivación: ¿Cómo lograr que el estudio de lenguajes de programación sea atractivo y constante, evitando las altas tasas de deserción en los niveles iniciales?
- Seguridad: ¿Cómo permitir que los estudiantes ejecuten código de manera abierta sin poner en riesgo la integridad y estabilidad de los servidores de la plataforma?

3.2 Consideraciones de hardware y software

3.2.1 Inventario disponible

Para el desarrollo y gestión del proyecto, el equipo de 3 personas cuenta con:

- Equipos de cómputo: 3 Laptops/PCs de desarrollo (Procesador ≥ i5/Ryzen 5, 16 GB RAM, SSD 256 GB, Windows 11 / Linux).
- Dispositivos de prueba: Smartphones Android (versión 10+) para validar la responsividad del editor de código en dispositivos móviles.
- Conectividad: Conexión a Internet de fibra óptica ≥ 100 Mbps para despliegue y trabajo colaborativo en la nube.
- Cuentas de servicios: Cuentas institucionales y de desarrollador en GitHub y Google Cloud.

3.2.2 Tecnología propuesta

Dada la complejidad del motor de ejecución de código, se ha seleccionado un stack que prioriza el aislamiento y la seguridad:

- Backend: Node.js (Express). Se elige por su eficiencia en el manejo de peticiones asíncronas hacia el Sandbox.
- Frontend: React (Vite) para una interfaz de usuario rápida y dinámica. Se integrará la librería Monaco Editor (el motor de VS Code) para la experiencia de codificación.
- Base de Datos: PostgreSQL (alojado en Supabase) para la gestión de usuarios, persistencia de progreso (XP) y almacenamiento de retos.
- Motor de Ejecución (Sandbox): * Opción A (Rápida): Integración con la API de Piston o Judge0 para procesamiento remoto.
    - Opción B (Segura): Contenedores Docker locales con límites de recursos (256 MB RAM / 10s CPU) para evitar bucles infinitos.
- Infraestructura: * Hosting Frontend: Vercel (ideal para despliegues rápidos en cada hito).
    - Backend/API: AWS Lightsail o DigitalOcean Droplet (Ubuntu LTS) para gestionar el motor de ejecución de código.

- Gestión del Proyecto: * Control de Versiones: GitHub (uso de Pull Requests para revisión de código entre los 3 integrantes).
    - CI/CD: GitHub Actions para el despliegue automático a los entornos de prueba antes de cada presentación mensual.
    - Tablero de Tareas: Notion o Trello bajo metodología Kanban (To Do / Doing / Done).
- Seguridad: Implementación de tokens para sesiones de usuario y certificados SSL de Let's Encrypt para cifrado de datos en tránsito.

4. Estudio de Factibilidad

4.1 Factibilidad Técnica

El análisis de factibilidad técnica confirma que el equipo de desarrollo cuenta con las herramientas de hardware, el stack de software y la infraestructura de red necesarios para cumplir con los requerimientos de CodeBreaker en el cronograma establecido.

4.1.1 Configuración de Hardware y Recursos Físicos

El proyecto es viable dado que la arquitectura PERN (PostgreSQL , Express, React, Node) es ligera y la carga de procesamiento pesada se delega a servicios externos:

- Servidor de Producción (Cloud): Se implementará un Servidor VPS (AWS Lightsail, DigitalOcean o Azure) configurado con 1–2 vCPU y 2GB-4GB de RAM. Esta potencia es más que suficiente, ya que el backend en Node.js solo actúa como orquestador y la ejecución de código de los estudiantes es procesada por la infraestructura de la API de Piston.
- Estaciones de Trabajo: El equipo dispone de laptops con capacidad para ejecutar Visual Studio Code y el entorno de ejecución de Node.js. Al no requerir virtualización local pesada (Docker) para el funcionamiento del sistema, se optimiza el tiempo de desarrollo.
- Infraestructura de Red: Se cuenta con una conexión de alta velocidad necesaria para el consumo asíncrono de la API de Piston y la carga de librerías desde CDNs (Tailwind CSS, Google Fonts).
- Dispositivos de Prueba: Se dispone de computadoras de escritorio y dispositivos móviles para validar la responsividad del framework Tailwind CSS y asegurar que el Monaco Editor sea funcional en diferentes resoluciones.

4.1.2 Stack de Software y Aplicabilidad

Se han seleccionado tecnologías modernas y estandarizadas que garantizan la compatibilidad, seguridad y rapidez de desarrollo para un equipo ágil:

- Frontend (Capa de Presentación): Se utilizará React.js en conjunto con Tailwind CSS. Esta combinación permite construir una interfaz altamente dinámica y responsiva, facilitando la integración de Monaco Editor para proporcionar una experiencia de codificación similar a la de un entorno de escritorio, pero totalmente basada en el navegador.
- Backend (Capa de Lógica): El servidor se desarrollará sobre Node.js utilizando el framework Express. Esta elección es estratégica debido a su modelo de entrada/salida no bloqueante, lo cual es ideal para gestionar las peticiones asíncronas de ejecución de código y la comunicación en tiempo real con servicios externos.
- Gestión de Datos: Se empleará PostgreSQL como sistema de gestión de base de datos relacional. Su robustez garantiza la integridad de la persistencia del progreso académico, la acumulación de puntos de experiencia (XP) y el registro histórico de logros de cada estudiante.
- Seguridad y Ejecución (Sandbox): La plataforma delega la ejecución de scripts a la API de Piston. Esta arquitectura permite procesar lenguajes como Python y JavaScript en entornos virtualizados aislados, asegurando que el código del estudiante no tenga acceso a la infraestructura del servidor principal y respetando límites estrictos de consumo de memoria y tiempo de CPU.
- Animaciones e Interacción: Se integrará Framer Motion para gestionar la capa de movimientos de la interfaz, asegurando que la retroalimentación visual de los logros y cambios de nivel sea fluida y mejore la retención del usuario.

4.1.3 Infraestructura de Red y Disponibilidad

Para garantizar que la plataforma sea accesible, segura y cumpla con los estándares de disponibilidad del 99.9%, se define lo siguiente:

- Dominio y Seguridad SSL: Implementación de un dominio profesional con certificado SSL/TLS (Let's Encrypt). Esto es crítico para asegurar que los JSON Web Tokens viajen de forma cifrada entre el navegador del estudiante y el backend en Node.js, evitando la interceptación de sesiones.
- Conectividad y Latencia: Uso de un Hosting Cloud (VPS) con una ubicación geográfica estratégica (preferencia en regiones de Sudamérica o EE. UU. Este) para minimizar el ping. Esto garantiza que la comunicación con la API de Piston y la devolución de resultados al Monaco Editor se perciban de forma casi instantánea.
- Gestión de Carga Asíncrona: Gracias a la naturaleza no bloqueante de Node.js, la infraestructura de red podrá gestionar múltiples peticiones simultáneas de ejecución de código sin degradar el rendimiento de la interfaz de usuario en React.

- CDN (Content Delivery Network): Se utilizarán redes de distribución de contenido para servir los assets estáticos de la plataforma (imágenes, scripts de React y estilos de Tailwind CSS), reduciendo el tiempo de carga inicial para los estudiantes en Tacna y otras regiones.

4.1.4 Metodología de Implementación Técnica

La metodología incremental garantiza que la complejidad técnica se aborde por etapas, priorizando la conectividad con el motor de ejecución:

- Hito 1 (Abril): Configuración del entorno Node.js, despliegue de la base de datos PostgreSQL y validación de la conexión asíncrona con la API de Piston.
- Hito 2 (Mayo): Integración del Monaco Editor en la interfaz de React, desarrollo de la lógica de evaluación (inputs/outputs) y gestión de sesiones mediante.
- Hito 3 (Junio): Implementación de animaciones con Framer Motion, optimización responsiva con Tailwind CSS para móviles y cierre de auditoría de seguridad.

Especificaciones Técnicas

Componente	Especificación Mínima	Sustento Técnico
Procesador	Intel Core i5 (8va Gen o superior)	Suficiente para la ejecución del entorno de desarrollo Node.js y la transpilación de código en React.
Memoria RAM	8 GB - 12 GB DDR4	Permite la ejecución fluida de VS Code, el navegador para pruebas de interfaz y el servidor local de desarrollo de forma simultánea.
Almacenamiento	SSD 256 GB	Garantiza un acceso rápido a los módulos de npm y reduce los tiempos de construcción (build) de la aplicación frontend.
Servidor Cloud	VPS (1–2 vCPU, 2GB-4GB RAM)	Infraestructura ligera en la nube necesaria para orquestar peticiones, ya que la ejecución pesada de código se delega a la API de Piston.
Dispositivos	Android 10+ / iOS 14+	Terminales requeridos para validar que el diseño basado en Tailwind sea funcional y el editor sea usable en pantallas táctiles.

Tabla 1. *Especificaciones de Hardware para el Proyecto*

Recurso	Definición / Uso en el Proyecto	Sustento Técnico
Visual Studio Code	IDE principal de desarrollo.	Estándar de la industria por su ligereza y soporte avanzado para extensiones de Node.js, JavaScript y Python.
Node.js (LTS)	Entorno de ejecución para el Backend.	Permite una gestión asíncrona y escalable de las peticiones mediante un modelo de I/O no bloqueante, ideal para servicios de micro-ejecución.
React.js (Vite)	Librería de Frontend (SPA).	Facilita la creación de una interfaz reactiva y rápida que permite ver los resultados del código y las animaciones de XP sin recargar la página.
PostgreSQL	Motor de Base de Datos Relacional.	Garantiza la integridad de los datos de progreso y niveles, ofreciendo una persistencia robusta y eficiente para un equipo de desarrollo ágil.
Monaco Editor	Motor de edición de código.	Proporciona resaltado de sintaxis y autocompletado profesional, ofreciendo al estudiante una experiencia de desarrollo industrial en el navegador.
Piston API	Motor de Ejecución (Sandbox).	Aísla el código de los usuarios en entornos virtuales seguros, limitando el uso de CPU y RAM para evitar ataques o bucles infinitos en el servidor.
Tailwind CSS	Framework de Estilos.	Permite un diseño responsivo acelerado y optimizado, asegurando que la plataforma sea usable en dispositivos móviles y laptops de bajos recursos.
GitHub / Kanban	Gestión y Control de Versiones.	Asegura la trazabilidad del código entre los integrantes y el cumplimiento estricto de los hitos de abril, mayo y junio.

Tabla 2. *Características de Software (Stack Tecnológico)*

4.2 Factibilidad Económica

Alcance del estudio: Análisis técnico, económico, operativo, legal, social y ambiental para determinar la viabilidad del MVP de CodeBreaker (entorno de ejecución de código, sistema de retos, gamificación y gestión de usuarios).

Actividades realizadas: Levantamiento de requerimientos, diagnóstico “as-is / to-be” de la enseñanza de programación, estimación de costos (infraestructura de servidores para Sandbox, personal, operación), definición de beneficios por suscripciones y convenios, evaluación financiera (B/C, VAN, TIR) y análisis de riesgos.

Aprobación: Revisado por el Equipo del Proyecto y validado académicamente por el docente Ing. Patrick Jose Cuadros Quiroga.

4.2.1 Costos Generales

Una tabla de costos generales de recursos de oficina presenta el desglose de inversión requerida para equipar una estación de trabajo completa, considerando los componentes esenciales para el funcionamiento operativo del sistema.

Recurso	Cantidad	Costo
Laptops de desarrollo (i5/16GB RAM/SSD)	3 unidad	S/. 9,600
Kit de periféricos (Mouse, Teclado, etc)	3 sets	S/. 360
Mobiliario de oficina	3 unidad	S/. 3,050.00
Total		S/. 11,010

4.2.2 Costos operativos durante el desarrollo

Durante el periodo de implementación, se consideran costos adicionales para asegurar la continuidad operativa:

Una tabla de costos operativos describe los gastos recurrentes necesarios para mantener la infraestructura y servicios durante el período de desarrollo del sistema, considerando los recursos esenciales para el funcionamiento del proyecto.

Concepto	Costo Mensual	Duración	Costo Total
Servicios básicos (Luz, agua, internet de alta velocidad)	S/. 350	3.5 meses	S/. 1,225
Soporte técnico y licencias (Suscripciones de software o soporte especializado)	S/. 200	3 meses	S/. 700
Renta de espacio de trabajo (Valorizado del lugar donde se reúnen a programar)	S/. 800	3 meses	S/. 2,450
TOTAL OPERATIVOS			S/. 4,375

4.2.3 Costos del ambiente

Es el costo de mantener la plataforma CodeBreaker disponible en internet para que los usuarios accedan al entorno de aprendizaje, ejecuten código y registren su progreso.

Concepto	Duración	Costo (S/.)
Dominio Web	4 meses	S/. 110.00
Servidor en la nube (Render/Railway para el backend y BD SQL)	4 meses	S/. 540.00
TOTAL AMBIENTE		S/. 650

4.2.4 Costos de personal

Correspondientes al recurso humano necesario para el desarrollo e implementación del sistema, incluyendo:

- Desarrolladores de software (backend y frontend)
- Tester o QA
- Diseñador UI/UX

Rol	Integrante	Horas / Semana	Costo por Hora (S/.)	Pago Mensual (S/.)	Meses	Subtotal (S/.)
Líder de Proyecto / Analista	Huaman Rivera	20 h	3.75	300.00	3.5	1,050.00
Desarrollador Backend	Huaman Rivera	30 h	3.33	400.00	3.5	1,400.00
Desarrollador Frontend	Rosas Chambilla, Jefferson	30 h	3.00	360.00	3.5	1,260.00
Tester / QA	Challo Coaquera	20 h	2.50	200.00	3.0	600.00
Bonificación / Ajuste técnico del proyecto	Equipo completo	—	—	—	—	6,540.00
TOTAL PERSONAL						S/. 10,850.00

4.2.5 Costos totales del desarrollo del sistema

Sumando todos los costos involucrados en el desarrollo, el costo total estimado es el siguiente:

Concepto	Costo en Soles

Costos generales	S/. 11,010
Costos operativos durante el desarrollo	S/. 4,375
Costos del ambiente	S/. 650
Costos de personal	S/.10,850.00
Costos totales	S/. 26,885

4.2.6 Evaluación Financiera

Para el cálculo de rentabilidad, se estima un modelo de monetización por suscripciones premium y convenios institucionales.

- Flujo de Caja Estimado (Año 1): S/ 12,000 (Ingreso neto tras gastos de mantenimiento).
- Flujo de Caja Estimado (Año 2): S/ 25,000.
- Flujo de Caja Estimado (Año 3): S/ 40,000.

Indicador	Resultado	Interpretación
VAN (Valor Actual Neto)	S/ 18,450.50	El proyecto es rentable al ser un valor positivo mayor a 0.
TIR (Tasa Interna de Retorno)	24%	La rentabilidad supera ampliamente el costo de oportunidad del capital (12%).
Periodo de Recuperación	1.6 años	La inversión se recupera en menos de 20 meses de operación.

4.2.7 Análisis de Sensibilidad

En un escenario pesimista (caída del 25% en las ventas de suscripciones), el VAN permanece en terreno positivo (S/ 4,100 aprox.), lo que demuestra que el proyecto es resiliente. El punto de equilibrio se alcanzaría en el mes 10 del primer año de operación comercial, asegurando la continuidad de CodeBreaker incluso ante variaciones negativas del mercado.

4.3 Factibilidad Operativa

El estudio de factibilidad operativa determina si el equipo cuenta con las capacidades técnicas y los procesos necesarios para asegurar el correcto funcionamiento, adopción y mantenimiento de CodeBreaker

4.3.1 Estructura Organizacional y Roles del Equipo

Dado que el equipo está compuesto por 3 integrantes, se han distribuido las responsabilidades para cubrir el ciclo de vida de desarrollo de software (SDLC) y la operación continua del sistema:

Rol	Responsabilidades Operativas
Líder de Proyecto / Backend	Gestión del servidor VPS, configuración de la arquitectura asíncrona en Node.js, administración de la base de datos PostgreSQL y supervisión de la conectividad con la API de Piston.
Desarrollador Frontend / UX	Mantenimiento de la SPA en React, optimización del Monaco Editor, implementación de animaciones con Framer Motion y asegurar la responsividad con Tailwind CSS.
Analista QA / Gestor de Contenido	Creación y validación lógica de los retos de programación (Python/JavaScript), pruebas de seguridad de los inputs y gestión del catálogo de niveles en la base de datos.

4.3.2 Análisis de Aceptación del Usuario

La plataforma ha sido diseñada bajo principios de UX (User Experience) modernos para minimizar la curva de aprendizaje, lo cual asegura su viabilidad operativa:

- Eliminación de Barreras Técnicas: Al ser una plataforma SPA (Single Page Application) accesible desde el navegador, el estudiante no requiere instalar software complejo (compiladores, variables de entorno o IDEs pesados). Esta "fricción cero" es el principal motor de aceptación operativa del sistema.
- Interfaz Profesional e Intuitiva: La integración de Monaco Editor proporciona un entorno familiar para quienes ya conocen herramientas como VS Code, mientras que la retroalimentación inmediata (resultados de ejecución y XP) refuerza el ciclo de aprendizaje positivo.
- Autogestión del Contenido Académico: Se incluirá un panel administrativo que permite al equipo docente cargar nuevos retos, definir los casos de prueba (inputs/outputs) y monitorear el progreso de los usuarios directamente en la base de datos PostgreSQL , sin necesidad de modificar o recompilar el código fuente en Node.js.
- Soporte y Mantenimiento Simplificado: Al delegar la ejecución del código a la API de Piston, el equipo operativo no tiene que preocuparse por parches de seguridad de lenguajes específicos (Python/JavaScript) o la gestión de contenedores Docker, reduciendo la carga de mantenimiento preventivo a nivel de servidor.

4.3.3 Procesos de Soporte y Mantenimiento

Para garantizar la alta disponibilidad del sistema y la continuidad del aprendizaje, se han establecido los siguientes procesos operativos:

- Monitoreo de Servicios Críticos: Uso de herramientas de monitoreo (como UptimeRobot o PM2 Monitor) para alertar al equipo en tiempo real sobre el estado del backend en Node.js y la latencia en la conexión con la API de Piston.
- Ciclo de Actualización Incremental: Las actualizaciones de la interfaz y parches de seguridad de las dependencias de npm se realizarán mensualmente, siguiendo la metodología validada en los hitos de abril, mayo y junio.
- Respaldo de Información (Backup): Copias de seguridad automáticas de la base de datos PostgreSQL para asegurar que el progreso de los estudiantes, sus puntos de XP y los logros alcanzados no se pierdan ante fallos de infraestructura.
- Gestión de Incidentes de Ejecución: Debido a que el Sandbox es externo, el soporte operativo se centrará en gestionar los timeouts y asegurar que los límites de memoria (256MB) sean suficientes para los ejercicios propuestos por los docentes.

4.3.4 Matriz de Impacto Operativo

Factor	Impacto	Estrategia de Mitigación
Capacitación	Bajo	La interfaz basada en React es intuitiva y sigue patrones de diseño modernos. Se incluirá un "Tour Guiado" dinámico y una sección de ayuda rápida para nuevos estudiantes.
Carga de Trabajo Administrativa	Medio	Uso de la API de Piston y automatización de la calificación mediante la comparación de outputs esperados. Esto reduce drásticamente la intervención manual de los 3 integrantes en la revisión de código.
Resistencia al Cambio	Mínimo	La integración de Framer Motion para la gamificación (Hito Junio) incentiva el uso constante de la plataforma. El estudiante percibe el aprendizaje como un reto lúdico frente a los métodos de enseñanza tradicionales.
Mantenimiento Técnico	Bajo	Al delegar la ejecución segura de Python/JavaScript a un servicio externo, el equipo operativo no requiere gestionar parches de compiladores locales, enfocándose solo en la estabilidad del backend en Node.js.

4.4 Factibilidad Legal

Este estudio analiza la viabilidad del proyecto CodeBreaker en relación con el marco jurídico vigente en el Perú, asegurando que la plataforma opere dentro de la legalidad y proteja tanto la propiedad intelectual del equipo como la privacidad de los usuarios.

4.4.1 Protección de Datos Personales

Al ser una plataforma que requiere el registro de usuarios (nombre, correo, progreso académico), CodeBreaker debe cumplir estrictamente con la Ley N° 29733 (Ley de Protección de Datos Personales) y su Reglamento:

- Finalidad y Consentimiento: Se implementará un aviso de "Términos y Condiciones" y "Políticas de Privacidad" donde el usuario acepte de forma libre y expresa el tratamiento de sus datos.
- Seguridad: El uso de certificados SSL (mencionado en la factibilidad técnica) garantiza la protección de la información en tránsito, cumpliendo con las medidas de seguridad exigidas por la Autoridad Nacional de Protección de Datos Personales (ANPD).

4.4.2 Propiedad Intelectual y Derechos de Autor

El desarrollo del software se rige por el Decreto Legislativo N° 822 (Ley sobre el Derecho de Autor):

- Autoría: El código fuente, la arquitectura del sistema y el diseño de la interfaz son propiedad intelectual de los 3 integrantes del equipo desarrollador.
- Uso de Terceros: La integración de bibliotecas de código abierto (como Monaco Editor, React o Node.js) se realizará bajo sus respectivas licencias (MIT, Apache 2.0), las cuales permiten su uso y distribución sin infringir derechos de autor.
- Contenido Educativo: Todos los retos y ejercicios de programación serán de autoría propia para evitar plagio de otras plataformas educativas.

4.4.3 Responsabilidad Civil y Ejecución de Código (Sandbox)

Dado que la plataforma permite la ejecución de código por parte de terceros (usuarios), legalmente es necesario establecer un Deslinde de Responsabilidad (Disclaimer):

- El sistema limitará la ejecución de comandos que puedan ser considerados ataques cibernéticos según la Ley N° 30096 (Ley de Delitos Informáticos).

- Se especificará que el equipo no se hace responsable por el uso indebido que los usuarios den a las herramientas de codificación dentro del entorno controlado del Sandbox.

4.4.4 Cumplimiento Tributario y Formalización

Aunque el proyecto se encuentra en fase de MVP (Hitos de Abril-Junio), su escalabilidad contempla:

- Formalización: En caso de monetización (visto en la factibilidad económica), el proyecto se registrará bajo el régimen MYPE tributario.
- Dominio .pe: El registro del dominio se realizará ante la Red Científica Peruana (RCP), cumpliendo con la normativa de registro de nombres de dominio en el Perú.

Matriz de cumplimiento legal

Norma / Ley	Ámbito de Aplicación	Acción de Cumplimiento
Ley N° 29733	Datos de usuarios	Políticas de privacidad y encriptación SSL.
D.L. N° 822	Código y Diseño	Registro de propiedad intelectual del equipo.
Ley N° 30096	Seguridad Informática	Implementación de Sandbox para evitar delitos informáticos.
Licencias MIT/OSI	Librerías Externas	Uso correcto de frameworks de código abierto.

4.5 Factibilidad Social

El estudio de factibilidad social evalúa cómo CodeBreaker contribuye al desarrollo de capacidades humanas y si existe una aceptación por parte de la comunidad objetivo. Este proyecto se alinea con la democratización del acceso a la educación tecnológica.

4.5.1 Democratización del Aprendizaje Técnico

Uno de los mayores impactos sociales de CodeBreaker es la eliminación de la brecha de hardware.

- Al procesar el código en la nube (vía Sandbox), el estudiante no necesita una computadora costosa para practicar Python o JavaScript .
- Permite que estudiantes con equipos de bajos recursos o que solo cuentan con dispositivos móviles puedan desarrollar habilidades de programación competitivas.

4.5.2 Reducción de la Curva de Frustración

Socialmente, la programación es percibida como una disciplina "excluyente" o "muy difícil".

- Impacto Positivo: Al eliminar la configuración técnica inicial (instalación de compiladores), el proyecto reduce la tasa de abandono en los primeros niveles de estudio.
- Motivación: La integración de gamificación (XP y logros) fomenta una cultura de perseverancia y autodesarrollo, transformando el aprendizaje en una actividad lúdica y socialmente atractiva.

4.5.3 Alineación con los Objetivos de Desarrollo Sostenible (ODS)

El proyecto contribuye directamente a los objetivos globales de desarrollo:

- ODS 4 (Educación de Calidad): Al proporcionar una herramienta interactiva que mejora la retención de conocimientos.
- ODS 8 (Trabajo Decente y Crecimiento Económico): Al preparar a jóvenes con habilidades tecnológicas que tienen alta demanda laboral, facilitando su futura inserción en el mercado de trabajo.

Matriz de Impacto Social

Grupo de Interés	Impacto Esperado	Beneficio Social
Estudiantes	Acceso inmediato a la práctica.	Empoderamiento técnico y mejora en la empleabilidad.
Docentes	Herramienta de apoyo didáctico.	Reducción de carga en la revisión manual de ejercicios básicos.
Comunidad Académica	Fomento de la innovación.	Creación de una cultura de "aprender haciendo" (learning by doing).
Sociedad	Reducción de brecha digital.	Formación de nuevos cuadros técnicos en la región.

4.6 Factibilidad Ambiental

El estudio de factibilidad ambiental evalúa el impacto del proyecto sobre el entorno natural. Al ser una solución basada en software y computación en la nube,

CodeBreaker se posiciona como una alternativa ecológica frente a los métodos de enseñanza tradicionales.

4.6.1 Reducción del Consumo de Recursos Físicos

- Cero uso de papel: Al digitalizar guías de estudio, exámenes y ejercicios de programación, se elimina la necesidad de impresiones físicas, contribuyendo a la conservación de recursos forestales.
- Reducción de Huella de Carbono por Desplazamiento: Al permitir que el aprendizaje y la práctica se realicen de forma remota y virtual, se disminuye la necesidad de traslados físicos a laboratorios de cómputo, reduciendo indirectamente las emisiones de CO2.

4.6.2 Eficiencia Energética y Cloud Computing

- Optimización de Hardware: En lugar de mantener múltiples servidores físicos locales encendidos 24/7, el uso de infraestructura en la nube (AWS/Azure) permite que el consumo energético se optimice mediante la virtualización. Los proveedores cloud seleccionados cuentan con políticas de neutralidad de carbono.
- Vida Útil de Equipos: Al requerir poca potencia de procesamiento local (ya que el código se ejecuta en el servidor), CodeBreaker permite que laptops antiguas o de gama baja sigan siendo útiles, extendiendo su ciclo de vida y reduciendo la generación de chatarra electrónica (e-waste).

4.6.3 Gestión de Residuos Digitales

El equipo se compromete a realizar una limpieza periódica de datos obsoletos y optimización de código para reducir la carga de almacenamiento en los servidores, lo que se traduce en un menor consumo de energía en los centros de datos (Green Computing).

Matriz de Impacto Ambiental

Factor Ambiental	Efecto de CodeBreaker	Beneficio
Residuos Sólidos	Eliminación de exámenes y manuales impresos.	Reducción de desperdicio de papel y tinta.
Consumo Eléctrico	Uso de servidores escalables bajo demanda.	Optimización del consumo energético por usuario.
Residuos Electrónicos	Menor exigencia técnica al hardware del cliente.	Prolongación de la vida útil de computadoras viejas.

Emisiones CO2	Fomento del aprendizaje remoto.	Menos desplazamientos vehiculares a centros de estudio.

5. Análisis Financiero

El análisis financiero de CodeBreaker evalúa la viabilidad económica del proyecto basándose en la inversión necesaria para el desarrollo del MVP y las proyecciones de rentabilidad a corto y mediano plazo.

El costo total estimado para el desarrollo del sistema asciende a S/ 26,885. Este monto consolida los siguientes rubros:

- Costos Generales: S/ 11,010 destinados a equipamiento físico y mobiliario.
- Costos de Personal: S/ 10,850 para cubrir el equipo de desarrollo y gestión durante el periodo de implementación.
- Costos Operativos: S/ 4,375 que incluyen servicios básicos, soporte y licencias.
- Costos de Ambiente: S/ 650 para el mantenimiento del sistema en la nube (dominio y servidores).

La evaluación financiera se basa en un modelo de monetización por suscripciones y convenios institucionales. Los indicadores calculados son los siguientes:

- VAN (Valor Actual Neto): Se sitúa en S/ 18,450.50, lo que confirma que el proyecto genera valor económico positivo.
- TIR (Tasa Interna de Retorno): Alcanza un 24%, superando el costo de oportunidad del capital (12%).
- Periodo de Recuperación: La inversión inicial se recupera en 1.6 años (aproximadamente 19 meses).
- Relación Beneficio/Costo (B/C):

Considerando una inversión total de S/ 39,470 y beneficios proyectados acumulados de S/ 77,000 durante los tres primeros años de operación, se obtiene:

B/C = Beneficios / Costos = 77,000 / 39,470 ≈ 1.95

Resultado:
Al ser el ratio mayor a 1, se determina que el proyecto es rentable, generando aproximadamente S/ 1.95 por cada sol invertido, lo que indica una recuperación favorable de la inversión bajo un enfoque conservador.

- Valor Actual Neto (VAN):

Supuestos:
Se consideran los flujos netos proyectados para los tres primeros años:

- Año 1: S/ 12,000
- Año 2: S/ 25,000
- Año 3: S/ 40,000

Con una tasa de descuento del 10% anual.

Cálculo del valor presente (PV):

- PV Año 1: 12,000 / 1.10 = S/ 10,909.09
- PV Año 2: 25,000 / 1.10^2 = S/ 20,661.16
- PV Año 3: 40,000 / 1.10^3 = S/ 30,052.59

Suma PV beneficios: S/ 61,622.84

VAN = 61,622.84 − 39,470 = S/ 22,152.84

Resultado:
Dado que el VAN es positivo, el proyecto es financieramente viable, generando valor económico por encima de la inversión inicial.

- Tasa Interna de Retorno (TIR):

Evaluando el flujo de caja:

[-39,470; 12,000; 25,000; 40,000]

Se obtiene una TIR aproximada del 28%.

Resultado:
Al ser la TIR superior al costo de oportunidad del capital (10%), se confirma que el proyecto presenta una rentabilidad atractiva, incluso bajo un escenario conservador.

El análisis financiero demuestra que CodeBreaker es un proyecto viable y rentable, con una recuperación progresiva de la inversión y un crecimiento sostenido en los beneficios. Bajo un enfoque conservador, los indicadores VAN, TIR y B/C respaldan su implementación como una solución educativa sólida y sostenible en el tiempo.

5.1 Justificación de la Inversión

5.1.1 Beneficios del Proyecto

Beneficios Tangibles

- Incremento de Ingresos: Se proyecta un crecimiento del 30% anual en la captación de usuarios mediante el modelo de suscripciones premium y convenios con instituciones educativas.
- Ahorro Operativo en Infraestructura: Al centralizar la ejecución de código en la nube (Sandboxing), se estima un ahorro de S/ 5,000 anuales en comparación con el mantenimiento de servidores físicos locales o soporte técnico presencial.
- Optimización de Recursos Administrativos: Reducción de gastos en materiales didácticos físicos y gestión manual de evaluaciones, ahorrando aproximadamente S/ 3,000 al año.

Beneficios Intangibles

- Experiencia del Estudiante: Mejora significativa en la curva de aprendizaje gracias a la retroalimentación inmediata del editor de código.

- Reputación Académica: Posicionamiento de la plataforma como una herramienta innovadora en el sector EdTech (Educación Tecnológica).
- Seguridad y Confidencialidad: Garantía de un entorno seguro para la ejecución de scripts y cumplimiento estricto de la Ley de Protección de Datos Personales.
- Ventaja Competitiva: Diferenciación clara frente a métodos de enseñanza tradicionales que requieren instalaciones de software complejas.

5.1.2 Criterios de Inversión

5.1.2.1 Relación Beneficio/Costo (B/C)

Considerando una inversión total de S/ 39,470 y beneficios acumulados proyectados de S/ 77,000 en un horizonte de 3 años, se obtiene:

B/C = Beneficios / Costos = 77,000 / 39,470 ≈ 1.95

Resultado:
Dado que el ratio es mayor a 1, el proyecto es rentable, generando aproximadamente S/ 1.95 por cada sol invertido. Este resultado evidencia una recuperación favorable de la inversión bajo un enfoque conservador.

5.1.2.2 *Valor Actual Neto (VAN)*

Para el cálculo se ha considerado un horizonte de 3 años y una tasa de descuento (COK) del 10% anual.
Los flujos netos estimados son:

- Año 1: S/ 12,000
- Año 2: S/ 25,000
- Año 3: S/ 40,000

Cálculo del valor presente (PV) de los beneficios:

- PV Año 1: 12,000 / 1.10 = S/ 10,909.09
- PV Año 2: 25,000 / 1.10^2 = S/ 20,661.16
- PV Año 3: 40,000 / 1.10^3 = S/ 30,052.59

Suma PV beneficios: S/ 61,622.84

VAN = 61,622.84 − 39,470 = S/ 22,152.84

Resultado:
Dado que el VAN es positivo, se confirma que el proyecto es financieramente viable, ya que genera valor económico por encima de la inversión inicial.

5.1.2.3 *Tasa Interna de Retorno (TIR)*

La TIR es la tasa de rentabilidad anual promedio que genera el proyecto, considerando los flujos de caja futuros.

Período	Ingresos Brutos (S/)	Egresos Operativos (O&M)	Egresos de Inversión (S/)	Flujo de Caja Neto (S/)
Año 0 (Inversión)	S/ 0.00	S/ 0.00	(S/ 39,470.00)	(S/ 39,470.00)
Año 1	S/ 20,000.00	(S/ 8,000.00)	S/ 0.00	S/ 12,000.00
Año 2	S/ 35,000.00	(S/ 10,000.00)	S/ 0.00	S/ 25,000.00
Año 3	S/ 55,000.00	(S/ 15,000.00)	S/ 0.00	S/ 40,000.00

En este caso se consideran los flujos:

Año	Flujo de Caja (S/.)
0 (Inversión inicial)	−39,470
1	+12,000
2	+25,000
3	+40,000

El cálculo arroja una TIR aproximada del 28%.

Dado que la TIR (28%) es mayor que el costo de oportunidad del capital (COK = 10%), se concluye que el proyecto es rentable y financieramente atractivo.

A diferencia de escenarios optimistas, este análisis se basa en proyecciones conservadoras, lo que refuerza la viabilidad real del proyecto sin sobreestimar los ingresos.

6. Conclusiones

El análisis de factibilidad realizado para el desarrollo de la la plataforma educativa CodeBreaker permite concluir que el proyecto es viable y altamente recomendable desde múltiples perspectivas.

Conclusión de Factibilidad Técnica

El análisis técnico demuestra que el proyecto CodeBreaker es totalmente viable, ya que el equipo cuenta con las herramientas, conocimientos y tecnologías necesarias para su desarrollo. El uso de un stack moderno (Node.js, React y PostgreSQL) junto con la integración de un entorno de ejecución seguro (Sandbox) permite garantizar estabilidad, escalabilidad y seguridad en la ejecución de código. Además, la arquitectura basada en servicios en la nube reduce la carga operativa y facilita el despliegue progresivo del sistema.

Conclusión de Factibilidad Económica

Desde el punto de vista económico, el proyecto es rentable y sostenible. Los indicadores financieros obtenidos (VAN positivo, TIR superior al COK y B/C mayor a 1) evidencian que la inversión inicial puede recuperarse en un plazo razonable, generando beneficios progresivos en el tiempo. El análisis conservador de ingresos refuerza la credibilidad del proyecto, demostrando que incluso bajo condiciones moderadas, CodeBreaker mantiene su viabilidad financiera.

Conclusión de Factibilidad Operativa

El proyecto presenta una alta viabilidad operativa, ya que el equipo ha definido roles claros y procesos eficientes para el desarrollo, mantenimiento y soporte del sistema. La interfaz intuitiva, basada en tecnologías modernas, facilita la adopción por parte de los usuarios, reduciendo la necesidad de capacitación. Asimismo, la automatización de la evaluación de código y el uso de servicios externos disminuyen la carga operativa, permitiendo un funcionamiento continuo y eficiente.

Conclusión de Factibilidad Legal

El proyecto CodeBreaker es legalmente viable, ya que cumple con las normativas vigentes en el Perú relacionadas con la protección de datos personales, propiedad intelectual y seguridad informática. La implementación de mecanismos como cifrado de datos, políticas de privacidad y uso adecuado de licencias de software garantiza la protección de los usuarios y del producto desarrollado, minimizando riesgos legales futuros.

Conclusión de Factibilidad Social

Desde la perspectiva social, el proyecto tiene un impacto altamente positivo, al facilitar el acceso al aprendizaje de programación mediante una plataforma accesible y práctica. CodeBreaker contribuye a la reducción de la brecha digital y promueve el desarrollo de habilidades tecnológicas en estudiantes, alineándose con objetivos de educación de calidad y crecimiento profesional. Esto fortalece su aceptación y relevancia en el entorno académico y social.

Conclusión de Factibilidad Ambiental

El proyecto es ambientalmente viable, ya que al ser una solución digital basada en la nube, reduce significativamente el uso de recursos físicos como papel y equipos de alto consumo energético. Además, fomenta el aprendizaje remoto, disminuyendo la necesidad de

desplazamientos y, por ende, la huella de carbono. Esto posiciona a CodeBreaker como una alternativa sostenible frente a métodos tradicionales de enseñanza.